

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №4

Настройка Prometheus + Grafana для метрик. Интеграция с приложением

Цель занятия

Освоить базовую организацию мониторинга backend-приложения на Go с использованием Prometheus для сбора метрик и Grafana для их визуализации.

Задачи занятия

1. Изучить назначение метрик и их роль в сопровождении серверных приложений.
2. Понять различие между логированием и мониторингом на основе метрик.
3. Освоить подключение Prometheus-клиента к Go-приложению.
4. Научиться публиковать метрики приложения через HTTP-эндпоинт `/metrics`.
5. Настроить Prometheus для регулярного сбора метрик приложения.
6. Подключить Prometheus как источник данных в Grafana.
7. Построить базовые панели для визуализации количества запросов, ошибок и времени обработки.
8. Отработать анализ состояния приложения по метрикам.

Теоретическая часть

1. Что такое метрики и зачем они нужны

Метрики — это числовые показатели, описывающие состояние приложения и его поведение во времени. Для backend-приложения такими показателями могут быть количество входящих запросов, число ошибок, длительность обработки, объём памяти, нагрузка на процессор, число активных соединений.

В отличие от обычных логов, метрики удобны не для чтения каждой отдельной ситуации, а для наблюдения за системой в целом. По ним можно увидеть, растёт ли число ошибок, замедляется ли приложение, становится ли больше запросов, меняется ли поведение сервиса во времени.

Официальные материалы Prometheus описывают модель, в которой приложение публикует метрики, а Prometheus затем регулярно их считывает и хранит как временные ряды. ([Prometheus](#))

2. Чем метрики отличаются от логов

Логи отвечают на вопрос: **что именно произошло**.
Метрики отвечают на вопрос: **как система ведёт себя в целом**.

Например:

- лог покажет, что конкретный запрос `/students/15` завершился ошибкой;
- метрика покажет, что за последние 10 минут доля ошибок выросла с 1% до 15%.

Для сопровождения backend-приложения нужны оба инструмента. Логи помогают разбирать отдельные инциденты, а метрики позволяют вовремя увидеть проблему на уровне системы.

3. Что делает Prometheus

Prometheus — это система мониторинга, которая получает метрики от приложений не через отправку из приложения, а через регулярный опрос HTTP-эндпоинта. Конфигурация Prometheus строится через `scrape_configs`, где описываются задания сбора и целевые адреса. В официальной документации прямо сказано, что файл конфигурации определяет `scraping jobs` и их `instances`, а `scrape_config` описывает набор `targets` и параметры сбора. (Prometheus)

Для Go-приложений типичный сценарий такой:

- приложение публикует `/metrics`;
- Prometheus раз в несколько секунд или минут делает HTTP-запрос на этот адрес;
- полученные значения сохраняются как временные ряды;
- затем эти данные можно анализировать запросами PromQL и отображать в Grafana. (Prometheus)

4. Что делает Grafana

Grafana не собирает метрики сама по себе в этом сценарии, а подключается к Prometheus как к источнику данных и строит панели, графики и дашборды. В документации Grafana Prometheus указан как встроено поддерживаемый `data source`, а добавление источника выполняется через `Connections → Add new connection / Data sources`. (Grafana Labs)

Именно Grafana делает наблюдаемость наглядной: преподаватель или разработчик может видеть динамику нагрузки, ошибок и задержек в виде графиков, а не только в виде сырых чисел.

5. Какие метрики мы будем использовать

В этой работе будут использованы три базовые метрики приложения:

- **Counter** — общее число HTTP-запросов;
- **Counter** — число ошибочных запросов;
- **Histogram** — длительность обработки HTTP-запросов.

В клиентской библиотеке Prometheus для Go есть базовые metric primitives и HTTP exposition через promhttp, а автоматическая регистрация метрик может делаться через promauto. ([Go Packages](#))

Этих трёх метрик уже достаточно, чтобы увидеть базовое состояние сервиса:

- сколько запросов приходит;
- сколько из них завершается ошибкой;
- как быстро отвечает приложение.

6. Что мы сделаем в этой работе

В ходе практической работы будет создано Go-приложение с маршрутами:

```
GET /health
GET /students/{id}
GET /metrics
```

Маршрут /metrics будет отдавать метрики в формате Prometheus через promhttp.Handler(), который в официальной документации описан как способ экспонировать Prometheus metrics via HTTP на основе prometheus.DefaultGatherer. ([Go Packages](#))

Prometheus будет настроен на сбор метрик с этого приложения.

Grafana будет подключена к Prometheus и использована для построения простого дашборда.

Практическая часть

Общая идея проекта

В этой работе создаётся учебное приложение на Go и минимальная локальная инфраструктура мониторинга:

- Go-приложение на порту 8080;
- Prometheus на порту 9090;
- Grafana на порту 3000.
- Go-приложение экспортирует метрики.
- Prometheus их собирает.
- Grafana их визуализирует.

Рекомендуемая структура проекта

```
pz4-monitoring/  
  cmd/  
    server/  
      main.go  
  internal/  
    httpapi/  
      handler.go  
      middleware.go  
      response_writer.go  
    metrics/  
      metrics.go  
    student/  
      model.go  
      repo.go  
  monitoring/  
    prometheus.yml  
  go.mod
```

Шаг 1. Создание проекта

Откройте PowerShell и создайте папку проекта:

```
mkdir pz4-monitoring
cd pz4-monitoring
go mod init example.com/pz4-monitoring
mkdir cmd
mkdir cmd\server
mkdir internal
mkdir internal\httpapi
mkdir internal\metrics
mkdir internal\student
mkdir monitoring
```

Для macOS/Linux:

```
mkdir -p pz4-monitoring/cmd/server
mkdir -p pz4-monitoring/internal/httpapi
mkdir -p pz4-monitoring/internal/metrics
mkdir -p pz4-monitoring/internal/student
mkdir -p pz4-monitoring/monitoring
cd pz4-monitoring
go mod init example.com/pz4-monitoring
```

Шаг 2. Установка зависимостей

Установите клиентскую библиотеку Prometheus для Go:

```
go get github.com/prometheus/client_golang/Prometheus
go get github.com/prometheus/client_golang/prometheus/promauto
go get github.com/prometheus/client_golang/prometheus/promhttp
```

Официальная библиотека `client_golang` содержит `core instrumentation package`, реестр метрик и под-пакет `promhttp` для

HTTP exposition метрик. promauto предоставляет конструкторы с автоматической регистрацией в registry. ([Go Packages](#))

Шаг 3. Модель студента

Создайте файл `internal/student/model.go`:

```
package student

type Student struct {
    ID          int64  `json:"id"`
    FullName    string `json:"full_name"`
    Group       string `json:"group"`
    Email       string `json:"email"`
}
```

Шаг 4. Репозиторий с тестовыми данными

Создайте файл `internal/student/repo.go`:

```
package student

import "errors"

var ErrStudentNotFound = errors.New("student not found")

type Repo struct {
    data map[int64]Student
}

func NewRepo() *Repo {
    return &Repo{
        data: map[int64]Student{
```



```

        1: {
            ID:      1,
            FullName: "Иванов Иван Иванович",
            Group:    "ИБВО-01-25",
            Email:     "ivanov@example.com",
        },
        2: {
            ID:      2,
            FullName: "Петрова Мария Сергеевна",
            Group:    "ИБВО-02-25",
            Email:     "petrova@example.com",
        },
        3: {
            ID:      3,
            FullName: "Сидоров Алексей Андреевич",
            Group:    "ИБВО-03-25",
            Email:     "sidorov@example.com",
        },
    },
}

func (r *Repo) GetByID(id int64) (Student, error) {
    st, ok := r.data[id]
    if !ok {
        return Student{}, ErrStudentNotFound
    }
    return st, nil
}

```

Шаг 5. Создание метрик приложения

Создайте файл `internal/metrics/metrics.go`:

```
package metrics

import (
    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
)

var (
    HttpRequestsTotal = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "app_http_requests_total",
            Help: "Total number of HTTP requests",
        },
        []string{"method", "path"},
    )

    HttpErrorsTotal = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "app_http_errors_total",
            Help: "Total number of HTTP error responses",
        },
        []string{"method", "path", "status_code"},
    )

    HttpRequestDuration = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
```

```

        Name:
"app_http_request_duration_seconds",
        Help:      "HTTP request duration in
seconds",
        Buckets: prometheus.DefBuckets,
    },
    []string{"method", "path"},
)
)

```

Пояснение

Здесь используются:

CounterVec — счётчик запросов;

CounterVec — счётчик ошибок;

HistogramVec — распределение времени обработки.

Официальная библиотека Prometheus для Go предоставляет `metric primitives` и `registry`; `promauto` регистрирует метрики автоматически.

(Go Packages)

Шаг 6. Обёртка над ResponseWriter

Создайте файл `internal/httpapi/response_writer.go`:

```

package httpapi

import "net/http"

type LoggingResponseWriter struct {
    http.ResponseWriter
    statusCode int
}

```

```

func NewLoggingResponseWriter(w http.ResponseWriter)
*LoggingResponseWriter {
    return &LoggingResponseWriter{
        ResponseWriter: w,
        statusCode:      http.StatusOK,
    }
}

func (lrw *LoggingResponseWriter) WriteHeader(code int)
{
    lrw.statusCode = code
    lrw.ResponseWriter.WriteHeader(code)
}

func (lrw *LoggingResponseWriter) StatusCode() int {
    return lrw.statusCode
}

```

Шаг 7. Middleware для записи метрик

Создайте файл `internal/httpapi/middleware.go`:

```

package httpapi

import (
    "net/http"
    "strconv"
    "time"

    "example.com/pz4-monitoring/internal/metrics"
)

```

```

func MetricsMiddleware(next http.Handler) http.Handler
{
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        start := time.Now()

        lrw := NewLoggingResponseWriter(w)
        next.ServeHTTP(lrw, r)

        duration := time.Since(start).Seconds()
        path := r.URL.Path

        metrics.HttpRequestsTotal.WithLabelValues(r.Method,
path).Inc()

        metrics.HttpRequestDuration.WithLabelValues(r.Method,
path).Observe(duration)

        if lrw.StatusCode() >= 400 {
            metrics.HttpErrorsTotal.WithLabelValues(
                r.Method,
                path,
                strconv.Itoa(lrw.StatusCode()),
            ).Inc()
        }
    })
}

```

Пояснение

Middleware позволяет централизованно измерять запросы, не дублируя код в каждом обработчике. Такой подход близок к

рекомендациям по instrumenting Go applications и HTTP servers, где метрики добавляются на уровне приложения и HTTP-обработки. (Prometheus)

Шаг 8. HTTP-обработчики

Создайте файл `internal/httpapi/handler.go`:

```
package httpapi

import (
    "encoding/json"
    "net/http"
    "strconv"
    "strings"

    "example.com/pz4-monitoring/internal/student"
)

type Handler struct {
    repo *student.Repo
}

func NewHandler(repo *student.Repo) *Handler {
    return &Handler{repo: repo}
}

func (h *Handler) Health(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
            http.StatusMethodNotAllowed)
        return
    }
}
```

```

    }

    w.Header().Set("Content-Type", "application/json;
charset=utf-8")
    _ = json.NewEncoder(w).Encode(map[string]string{
        "status": "ok",
    })
}

func (h *Handler) GetStudentByID(w http.ResponseWriter,
r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    path := strings.TrimPrefix(r.URL.Path, "/students/")
    if path == "" || path == r.URL.Path {
        http.Error(w, "student id is required",
http.StatusBadRequest)
        return
    }

    id, err := strconv.Atoi(path, 10, 64)
    if err != nil {
        http.Error(w, "invalid student id",
http.StatusBadRequest)
        return
    }

    st, err := h.repo.GetByID(id)

```

```

        if err != nil {
            http.Error(w, "student not found",
http.StatusNotFound)
            return
        }

        w.Header().Set("Content-Type", "application/json;
charset=utf-8")
        _ = json.NewEncoder(w).Encode(st)
    }

```

Шаг 9. Точка входа приложения

Создайте файл `cmd/server/main.go`:

```

package main

import (
    "log"
    "net/http"

    "example.com/pz4-monitoring/internal/httpapi"
    "example.com/pz4-monitoring/internal/student"
    "github.com/prometheus/client_golang/prometheus/promh
ttp"
)

func main() {
    repo := student.NewRepo()
    handler := httpapi.NewHandler(repo)

    mux := http.NewServeMux()
    mux.HandleFunc("/health", handler.Health)

```



```
    mux.HandleFunc("/students/", handler.GetStudentByID)
    mux.Handle("/metrics", promhttp.Handler())

    rootHandler := httpapi.MetricsMiddleware(mux)

    log.Println("server started on :8080")

    if err := http.ListenAndServe(":8080", rootHandler);
err != nil {
        log.Fatal(err)
    }
}
```

Пояснение

`promhttp.Handler()` создаёт `http.Handler` для публикации метрик Prometheus через HTTP. Это прямо указано в документации пакета `promhttp`. ([Go Packages](#))

Шаг 10. Запуск приложения

В корне проекта выполните:

```
go run ./cmd/server
```

Откройте в браузере:

```
http://localhost:8080/metrics
```

Вы должны увидеть набор метрик в текстовом формате Prometheus. В официальных примерах по instrumenting Go application Prometheus также показывает метрики через HTTP endpoint, который затем опрашивается Prometheus. ([Prometheus](#))

Шаг 11. Проверка появления метрик

Сделайте несколько запросов:

```
curl http://localhost:8080/health
curl http://localhost:8080/students/1
curl http://localhost:8080/students/2
curl http://localhost:8080/students/999
```

Затем снова откройте `/metrics` и найдите строки, начинающиеся с:

```
app_http_requests_total
app_http_errors_total
app_http_request_duration_seconds
```

Шаг 12. Создание конфигурации Prometheus

Создайте файл `monitoring/prometheus.yml`:

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: "go_app"
    static_configs:
      - targets: ["localhost:8080"]

  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

Пояснение

Prometheus использует YAML-конфигурацию. В официальной документации показано, что конфигурация содержит `global` и `scrape_configs`, а в `static_configs` перечисляются цели сбора. Примеры с `targets: ["localhost:9090"]` и аналогичными записями приведены в руководствах Prometheus. ([Prometheus](#))

Шаг 13. Запуск Prometheus

Конкретная команда запуска зависит от того, как Prometheus установлен у вас: через архив, пакетный менеджер или Docker. Общий принцип подтверждается официальной документацией: Prometheus запускается бинарным файлом с указанием конфигурационного файла. ([Prometheus](#))

Пример для локального бинарного файла:

```
prometheus --config.file=monitoring/prometheus.yml
```

После запуска откройте:

```
http://localhost:9090
```

В интерфейсе Prometheus можно проверить, что target `go_app` находится в состоянии UP.

Шаг 14. Проверка целей сбора в Prometheus

Откройте в браузере раздел Targets.

Обычно это страница со списком jobs и targets, где видно состояние UP или DOWN. Если target `localhost:8080` доступен и маршрут `/metrics` отвечает, Prometheus должен собирать данные успешно. Это соответствует общему принципу scraping jobs и targets, описанному в документации. ([Prometheus](#))

Также в строке запроса Prometheus можно выполнить:

```
app_http_requests_total
```

и убедиться, что данные уже поступают.

Шаг 15. Запуск Grafana

Конкретный способ запуска Grafana тоже зависит от среды установки, но стандартный локальный доступ обычно идёт через порт 3000, а официальная документация Grafana описывает дальнейшую настройку источника данных через интерфейс Connections / Data sources. ([Grafana Labs](#))

После запуска откройте:

`http://localhost:3000`

Шаг 16. Подключение Prometheus в Grafana

Согласно официальной документации Grafana, чтобы добавить источник данных Prometheus, нужно:

1. Открыть **Connections**.
2. Перейти в **Data sources** или **Add new connection**.
3. Выбрать **Prometheus**.
4. Указать URL Prometheus.
5. Сохранить настройки. ([Grafana Labs](#))

Для локального запуска Prometheus URL обычно будет таким:

`http://localhost:9090`

[Вывод] В некоторых вариантах локальной установки, особенно через Docker, URL может отличаться, например использовать имя контейнера вместо localhost.

Шаг 17. Создание дашборда в Grafana

Создайте новый Dashboard и добавьте панели.

Панель 1. Общее число запросов

Пример запроса:

```
sum(app_http_requests_total)
```

Панель 2. Число ошибок

```
sum(app_http_errors_total)
```

Панель 3. Количество запросов по маршрутам

```
sum by (path) (app_http_requests_total)
```

Панель 4. Средняя длительность обработки

Для простого учебного сценария можно использовать упрощённый запрос на основе histogram sum/count:

```
sum(rate(app_http_request_duration_seconds_sum[1m]))  
/  
sum(rate(app_http_request_duration_seconds_count[1m]))
```

Это типичный способ оценить среднее время обработки по histogram-метрике в Prometheus.

Панель 5. Число ошибок по коду ответа

```
sum by (status_code) (app_http_errors_total)
```

Шаг 18. Генерация данных для графиков

Чтобы графики в Grafana начали меняться, выполните несколько запросов в цикле.

Для PowerShell:

```
1..20 | ForEach-Object { curl  
http://localhost:8080/health }  
1..15 | ForEach-Object { curl  
http://localhost:8080/students/1 }  
1..5 | ForEach-Object { curl  
http://localhost:8080/students/999 }
```

После этого обновите панели Grafana.

Вы должны увидеть:

- рост общего числа запросов;

- наличие ошибок 404;
- изменение статистики по маршрутам.

Что получилось в результате

В ходе практической работы:

реализовали экспорт метрик из Go-приложения;

подняли endpoint `/metrics`;

настроили Prometheus для регулярного сбора метрик;

подключили Prometheus к Grafana;

создали базовые панели мониторинга;

увидели, как метрики отражают поведение backend-приложения.

Что важно понять по итогам работы

Мониторинг — это не просто «ещё один инструмент». Для backend-разработки это способ видеть систему в динамике.

Если приложение начало отвечать медленнее, это можно увидеть по метрике времени обработки.

Если стало больше ошибок, это можно увидеть по счётчику ошибок.

Если выросла нагрузка на конкретный маршрут, это можно увидеть по разрезу по `path`.

Prometheus отвечает за сбор и хранение временных рядов, а Grafana — за их визуализацию и анализ. Это разделение ролей прямо соответствует официальной документации обеих систем. ([Prometheus](#))

Дополнительные задания

Вариант 1. Добавить метрику по студентам

Добавьте счётчик:

```
app_student_requests_total
```

с лейблом `student_id`.

Вариант 2. Добавить метрику времени только для маршрута `/students/{id}`

Реализуйте отдельную histogram-метрику для бизнес-маршрута.

Вариант 3. Добавить Gauge

Можно реализовать gauge-метрику для числа активных запросов, увеличивая значение в начале middleware и уменьшая в конце.

Вариант 4. Построить отдельный дашборд ошибок

Создайте дашборд только по ошибкам: число ошибок, ошибки по статусам, ошибки по маршрутам.

Контрольные вопросы

1. Что такое метрики приложения?
2. Чем метрики отличаются от логов?
3. Какую роль выполняет Prometheus?
4. Что такое scraping в Prometheus? ([Prometheus](#))
5. Зачем приложению маршрут `/metrics`? ([Go Packages](#))
6. Что делает `promhttp.Handler()`? ([Go Packages](#))
7. Для чего нужна Grafana?
8. Какие три основные метрики реализованы в этой работе?
9. Что показывает Histogram?
10. Почему мониторинг важен для сопровождения backend-приложений?

Типичные ошибки

Если `/metrics` не открывается, проверьте, что маршрут действительно подключён через `promhttp.Handler()` и приложение запущено. ([Go Packages](#))

Если Prometheus показывает target как DOWN, обычно проблема в неправильном адресе, порте или недоступности приложения. [Вывод] Также причиной может быть ошибка в `prometheus.yml`.

Если в Grafana не видно данных, проверьте, что Prometheus добавлен как источник данных и что в самом Prometheus запрос `app_http_requests_total` уже возвращает значения. ([Grafana Labs](#))

Если метрики есть, но графики пустые, возможно, вы ещё не сгенерировали достаточно запросов для накопления данных.

Если метрика с лейблом `path` даёт слишком много вариантов, это может быть проблемой дизайна метрик. [Вывод] В реальных системах часто нормализуют маршруты, чтобы вместо `/students/1` и `/students/2` использовать общий шаблон `/students/{id}`.